

CPSC 455 Project Design and Mockups: LoadLink

Team Name

Fantastic Five

Project Name

LoadLink

Team Members

Alexandar Lackovic (74213307)

Wendy Tso (34159368)

Max Yan (86293560)

Kyle Jones (82804451)

Eojin Lee (25508730)

Project Summary

Project Description

Truck driving management system designed to streamline the logistics of cargo transport through auction-based marketplace. Our platform allows companies to post freight requirements that drivers can accept through a bidding system.

Inspirations

- **Uber Courier:** Market place system connecting shippers and carriers directly
- **eBay:** Bidding system/reverse auction model, similar to how eBay handles competitive bidding, where drivers bid down the price to win contracts
- **Google Maps:** Displaying routes and location tracking
- **uShip:** Verified reviews and ratings in transport auctions, similar to their feedback system to encourage trust between shipper and driver

Target Users

- **Truck Drivers/Carriers:** Drivers will use the platform to find freight that matches their truck qualifications, using the bidding system to maximize their earnings.
- **Companies Looking To Ship Cargo:** companies use the platform to reduce manual dispatching and to find reliable transport, using the bidding system to maximize their savings.

Data

- **User Profiles (PII):** Things like names, contact information, company details, etc.
- **Truck and Truck Driver Data:** Truck dimensions, licensing information, driver certifications, reviews, etc.
- **Load Details:** Origin, destination, pricing, weight, commodity type, etc.
- **Transaction Data:** Bidding data, pricing details, revenue, expenses, etc.
- **Geospatial Data:** GPS location/coordinates, routing, etc.
- **Transit and Compliance Documents:** Auto-generated Rate Confirmations (contracts) upon auction completion, and image data for Bills of Lading (BOL) used for proof of delivery

Non-trivial Features

- **Time Decay Reverse Auction System**
 - The reverse auction system allows the drivers to work for the price that is fair to them and for companies to get the lowest cost possible to ship cargo
- **Map System**
 - Routes won via auction show up on the map dashboard, for a visual representation of pending work
 - Provide a suggested route for drivers to deliver goods.
 - Provide a way for companies to track the progress of a driver. Based on peer-feedback, we are moving away from persistent live-tracking and instead towards a check-in system.

- **Rating System/Preferences**
 - The companies or drivers should be able to see reviews of past performances so that they can make an informed decision before bidding or accepting a bid

Standard Features

- Secure user authentication for different roles: companies and drivers.
- Load posting (CRUD for freight)
- Search, filter, and pagination of loads
- Profile and fleet management (CRUD for users to update their truck specs and information)
- Bidding interface
- Notification system
- Dashboard Specific To Drivers
- Revenue Dashboard For Companies

Risks

- Location privacy as we're tracking drivers/freight. Based on peer-feedback, we are looking at scaling this down away from live-tracking and instead maybe on a check-in basis, where the driver's location is only updated upon visiting the web-app and only then.
- Data privacy with regards to personal information, transported goods, pay, etc.
- Data is hard to get and needs to be mocked. Eg: suggestion price, expense, revenue etc.
- Data accuracy of components like load weight/dimensions or hazardous materials to prevent safety issues
- Country risk. Based on peer-review feedback, there could be risks executing cross-border travel. For this MVP, we will focus on Canada-only travel.

Feedback

We received a lot of great feedback from our peers. We have actually changed the scope of our project slightly as a result, such as through the removal of a live-tracking aspect for drivers. We are also going to start small and ensure we can have the project running in just one country (Canada) before considering expanding it southward.

1. How does the server handle concurrent auctions? *Peer-provided feedback.*

- For our MVP, we will use a single master heartbeat engine that wakes up every second, queries any auctions that are due for a price change, updates them in the database, and instantly broadcasts the new prices to the drivers.
- If an auction is won such as by a driver clicking “claim the auction at the current price”, it will instantly send a network call to our server, which will log who claimed which auction, and then disseminate the closure of the auction to every user who is following the auction.

2. Consider handling different currencies for cross-border drives. *Peer-provided feedback.*

- This is a good question. In reality there are specific rules/regulations for couriers/drivers crossing the Canadian/USA border in addition to potential courier concerns. For the purposes of our MVP, we will focus only on the Canadian market (and therefore Canadian maps).

3. The rating system should have more categories to do the review: timeliness, communication and reliability. *Peer-provided feedback.*

- This is a good point. We will include numerous categories/aspects they can rate for, which will sum to a total rating for both companies and drivers, respectively.
- As an example, consider 5 categories. If a user is rated 5 on two categories, and 4 on three categories, that averages to a score of 4.4 / 5.

4. Will driver location tracking be live, or will it only show the planned route? *Peer-provided feedback.*

- This got us thinking about the routing system, in that it’s too complex and not core to our product idea. There are entire products focused around live tracking, which likely is way beyond the scope of our project idea.
- As a result, we are ultimately scaling back tracking. We may instead include a mechanism such that accessing the web app updates the location to the user’s current location, instead of constantly tracking them. Perhaps, as part of a stretch goal, we could require check-ins to the web-app every X period of time, which then updates the tracking information.

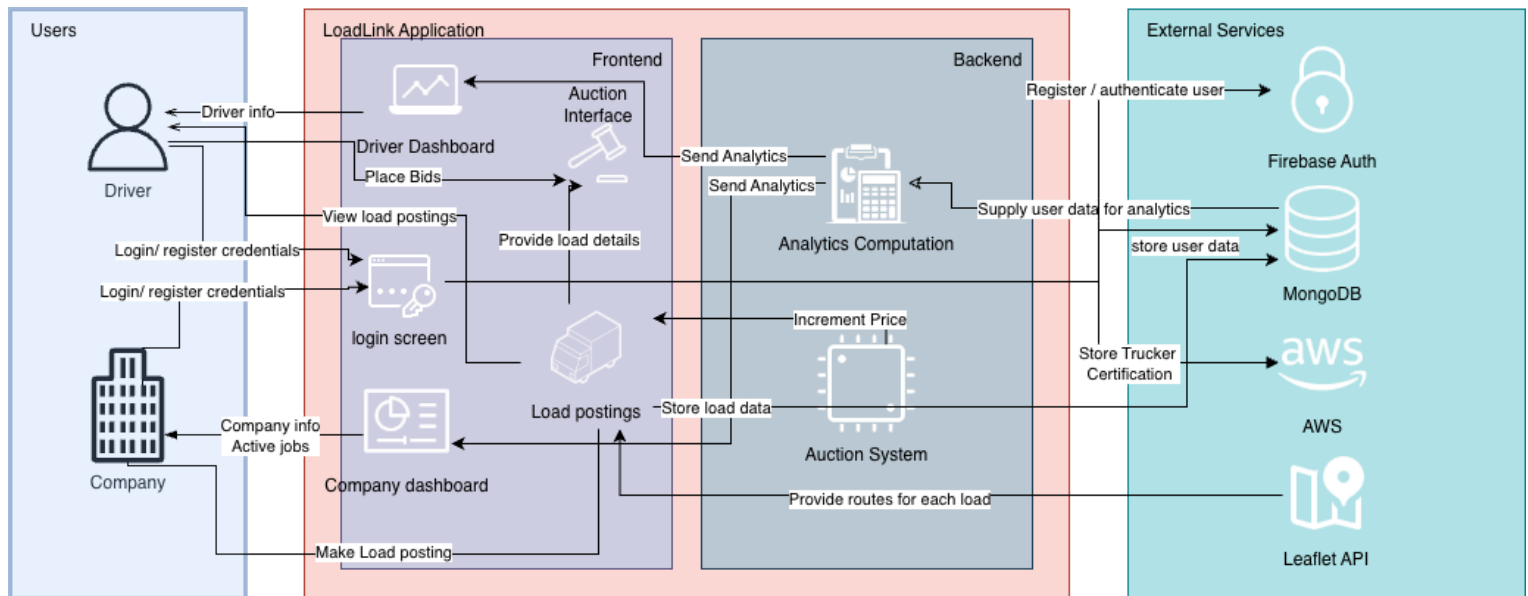
Highlighting a minor scope change

We will no longer be including a live tracking system, as indicated under feedback 4 above. It is not necessary to our core product. We may instead include location updating when drivers open the web application on a per-session basis.

Technical Design

Dataflow Diagram

The below diagram shows 2 types of users, Drivers and Companies, and how they interact with the application platform. Both user types begin by authenticating via the login/registration screen which communicates with Firebase Auth to register and validate the credentials. Once logged in, drivers access their dashboard to view load postings and place bids through the Auction Interface, while companies can access their dashboard to make load postings and view active jobs. On the back end, the Auction system is what processes the load data, does the reverse auction pricing system and provides route information by communication with Leaflet API. The Analytics Computation layer receives data to generate insights to be displayed on the dashboards on the frontend. Persistent data like load details, user profiles and trucker certifications is stored across MongoDB and AWS S2.



Tech Stack and Architecture

Component	Technology	Description
Frontend	React, RTK Query, Tailwind CSS with shadcn/ui or MUI, Mapbox	Used to build user interface for shippers and drivers
Backend	Node.js, Express.js	RESTful API + SSE (for bidding, price update, etc.)
Database	MongoDB Atlas and Mongoose ODM	Used to store data for loads, ratings, past jobs, etc
Maps API	Leaflet or OpenStreetMap or Mapbox	Provides route visualization and geocoding for load origin/destination points
Authentication	Firebase Authentication	Handles secure sign-up, login, and session management for different user roles (Shipper vs Driver)
CI/CD Pipeline	Github Actions	Handles test running and linter
Testing	Jest, React Testing Library	Use Jest for both backend and frontend testing (in combination with react testing library)
File Storage	Firebase Cloud or AWS S3	Handles document storage (PDFs, credentials, etc)
Hosting/Deployment	Vercel (frontend) and Heroku or AWS EC2 (backend)	Handles hosting for each component
Document Generation	pdf-lib (node.js)	Used by the backend to load a static Rate Con template and dynamically inject the finalized auction details (route, driver, rate, etc) into it

Main Systems and Services

LoadLink will utilize numerous concurrent and crucial systems. Among the most important is our real-time bidding system, which is crucial for ensuring the primary non-trivial aspect (the reverse auction for loads) works as intended.

Authentication System

Service: Firebase Authentication

APIs Used: Firebase Auth SDK

Functionality:

- Secure login for drivers, shippers, and LoadLink admin. As a stretch goal, the LoadLink admin will be able to review and approve driver and shipper documentation.
- Token-based authentication for API access
- Role-based access restrictions. Drivers and shippers do not see the same dashboards and bidding interfaces. Shippers cannot see the bids of other companies.

Load Management System

Service: Custom backend (Node.js + Express.js)

APIs Used:

- Custom-made REST endpoints (such as /loads, /loads/:id, /loads/:id/bids, etc.)
- Firestore or MongoDB SDK for persistence

Functionality:

- Create, update, and list freight loads
- Store load supplemental information (origin, destination, weight, type, hazard information, timing/duration, etc)
- Auto generates legal Rate Confirmation PDF contracts using pdf-lib when a reverse auction successfully ends. The resulting document is securely uploaded to AWS S3.

Real-Time Bidding System

Service: Custom SSE implementation

APIs Used: Custom SSE channels for updating price change and bids change

Functionality:

- Drivers place bids in real time, preventing stale/conflicting bids
- Shippers see live bid updates (gives shippers some flexibility if they want to expand the price ceiling)

Driver Location & Tracking System

Stretch Goal

Service: Custom SSE implementation for real-time location streaming, utilizing Mapbox or Leaflet + OpenStreetMap (OSM)

APIs Used: Mapbox Geocoding API (if we utilize Mapbox), OSM tile API (if we utilize Leaflet)

Functionality:

- Displays driver's live location for shipper to track progress
- Driver can view their progress in relation to where/when their next job will be

Mapping & Routing Service

We are targeting two possible third-party mapping APIs for the purposes of tracking origin, destination, and as a stretch goal, routing suggestions.

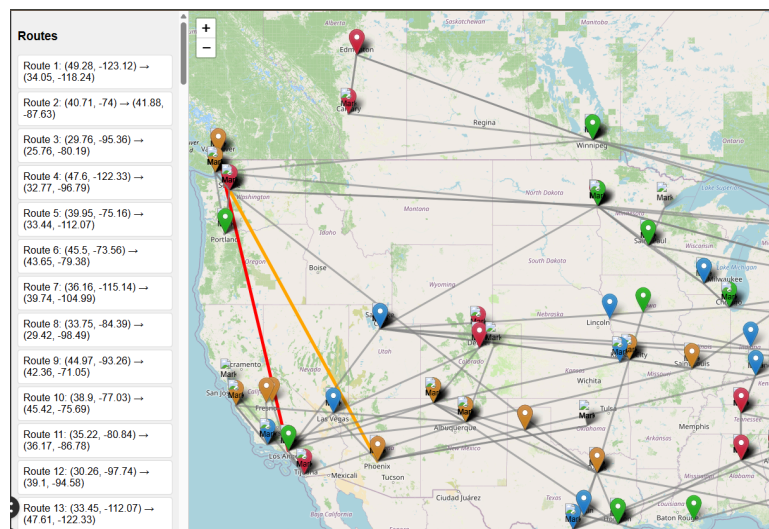
MapBox

Mapbox is the most comprehensive mapping service, including origin & destination points, as well as AI-assisted navigation to optimize routing. The maps can be heavily customized, including 3D modeling. While the routing would be great for our project, Mapbox is a freemium model where a credit card must be added to an account to access the “free” tier. Usage in the project could exceed the free tier. Given our use case of highlighting routes, 3D modeling (while cool) does not benefit our application.

Leaflet - Our Primary Source of Routing

Leaflet is an open-source framework to display maps via coordinates.

- It does not have any usage, request, or cost limits.
- It requires a tile-mapping service to function, where we can utilize OpenStreetMaps (OSM), a free mapping service for small/medium projects, without additional cost. OSM includes a geocoder built in (which converts language-based addresses into coordinates).
- In a cursory analysis (using AI generated code that will not be included in the project), we were able to specify origin/destinations, draw routes between them, highlight a route via clicking on either source, and so forth. OSM also includes possible routing, however it may not be sophisticated enough for the routing we have in mind.



Functionality:

- Convert addresses to coordinates for the purpose of visualizing routes, highlighting pick-up and drop-off points
- Drivers and shippers can view their current and historical routes
- (Stretch goal) suggesting optimized routes to line-up a driver's next job to their current job

Ratings & Reviews System

Service: Custom backend (Node.js + Express.js)

APIs Used:

- Custom-made REST endpoints (such as /ratings, /users/:id/ratings, etc.)
- Firestore or MongoDB SDK for persistence

Functionality:

- Drivers and shippers will be able to leave reviews/ratings on each other
- (Stretch goal) each side can manage a blacklist of shippers or drivers that do not want to see jobs from or for, respectively

File Storage System

Service: AWS S3

APIs Used: Standard upload and download APIs

Functionality:

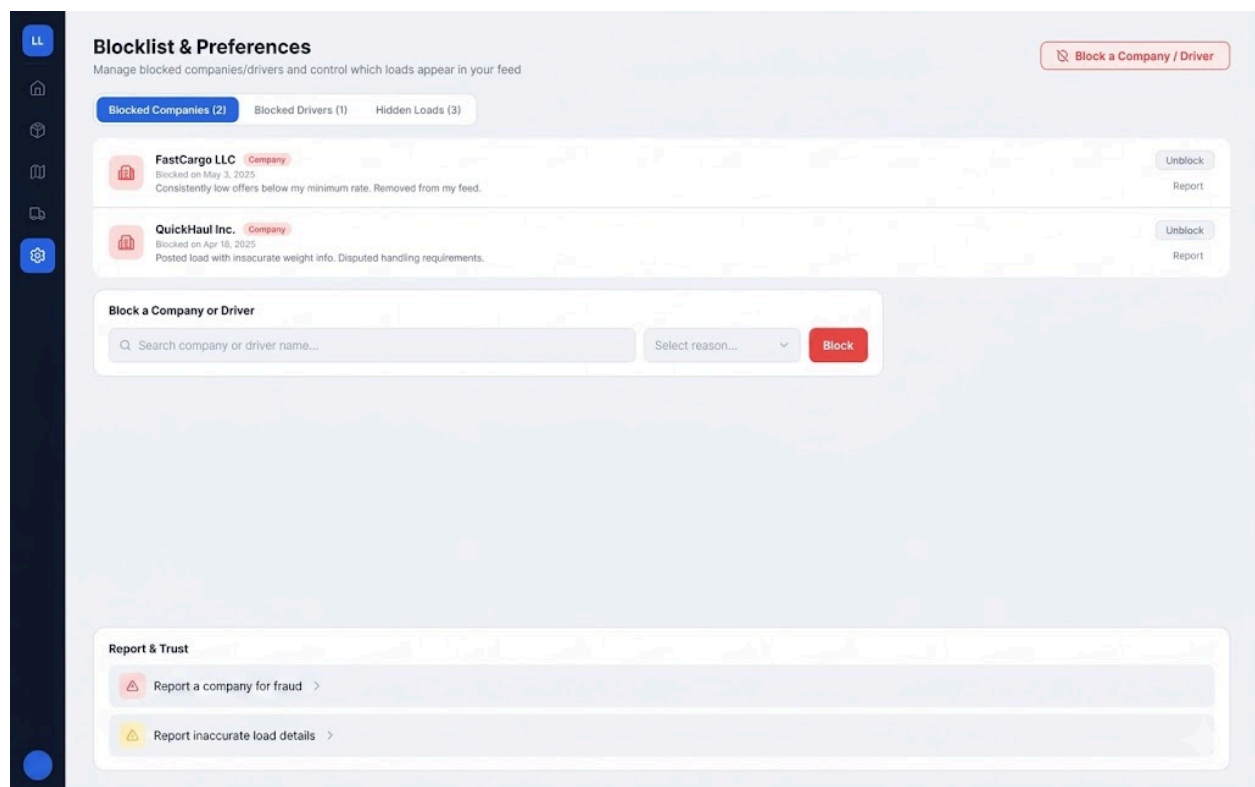
- Store documents in accordance with best privacy / security practices
- Documents include driver documents (licences, insurance, vehicle service report, etc.) and job-related documentation (rate confirmations, etc.)

User Interface (UI) Mockups

In the below section, you will see the mock-ups we have created for the UI. Disclosure: we fed our wireframes and description of our project into an AI UI-generating tool to create these mock-ups. They are in agreement with our vision, and are not reflections of the final version of our project.

Blocklist & Preferences Page

This page will be used by both companies and the drivers to manage their blocked entities. If a driver is blocked, their loads will be hidden from the driver. If a company is blocked, a driver will not see their open bids on the auction page. Given the platform is usable by professionals who must first provide their legal and valid credentials, this will prevent possible grieving by bad actors who intentionally sabotage the other side.



How to use:

1. **Tab navigation includes 3 tabs:** i) Blocked companies, ii) Blocked Drivers, and iii) Hidden Loads. The user clicks any tab to switch to that view
2. **Viewing blocked entries:** each card shows the company or driver name, a type badge (either company or driver), the date blocked, and the reason the user provided. Two action items on the right

- Unblock: removes the company/driver from the block list. The feed will begin showing their loads again
 - Report
3. **Blocking a new company or driver:** user can use the search bar at the bottom of the panel to find the company/driver by name then use the drop down “Select reason” to pick a reason (e.g. “Harrassment”, “Inaccurate info”)
 4. **Report forms (right sidebar):** user clicks on this button to open a fraud-report or inaccurate load details form.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Blocked companies tab	/users/{userId}/blocklist?type=company	GET	Fetches all companies the user has blocked to populate the Blocked Companies tab list
Blocked drivers tab	/users/{userId}/blocklist?type=drivers	GET	Fetches all drivers the user has blocked to populate the Blocked Drivers tab list
“Block” button (search + reason form drop down)	/users/{userId}/blocklist	POST	Adds the searched company/driver to the block list with the selected reason from the drop down
“Unblock” button on a blocked driver/company	/users/{userId}/blocklist/{blockId}	DELETE	Removes the driver/company from the blocklist
Report company for fraud button link	/reports	POST	Submits a fraud report against the selected company for review
Report inaccurate load details button link	/reports	POST	Submits an inaccuracy report against a specific load posting for review

Company Facing Auction Page

This is the auction page viewed by the company. The company can see the current price of their load and their pre-set algo. They can also extend the auction deadline, update the cap price and cancel the auction. On the right panel, they can view a list of bids sent by truckers and decide if they want to accept any of them.

The screenshot displays the 'Company Facing Auction Page' interface. On the left, the 'Active Loads / GREENLEAF - Auction Live' section shows load details for 'GREENLEAF N°1234789', a 'Frozen Produce - Reefer Van' with '55,000 lbs', originating from 'Newark, NJ' and heading to 'Chicago, IL (ORD)', with a distance of '620 mi' and a deadline of 'Jun 12-14'. The 'Auction Price Tracker' shows a price range from '\$800' to '\$1,000' with a current price of '\$847' and an auto-accept tolerance of 'Up to \$1,100 (10% above cap)'. The 'Auction Controls' section includes buttons for 'Accept Best Bid Now (\$1,050 — Alex M.)', 'Extend Deadline', 'Edit Cap Price', and 'Cancel Auction & Remove Load'. The right panel, titled 'Live Bids', shows '3 bids' and highlights the 'Best Bid Right Now' as '\$1,050 by Alex Martinez' with a rating of 4.9. Below this, the 'All Bids (3)' list shows three bids: Alex Martinez (\$1,050, Best), Sam Kowalski (\$980, Pending), and Raj Patel (\$850, Below Auto-accept).

How to use:

1. **Viewing load details and metrics:** the top-left area displays the company name, unique load ID, active status badge, elapsed time since posting, vehicle requirements, freight dimensions, commodity type, certifications, and target route distance.
2. **Tracking the reverse auction price:** the "Auction Price Tracker" section features a visual progress slider that highlights the starting price, max cap price, and auto-accept tolerance boundary. It shows the live, ticking current price alongside a badge displaying the exact rate of the preset price-creep algorithm.
3. **Viewing live driver bids:** the right panel displays all incoming preferred bids. The absolute highest current offer is pinned to the top in a green "Best Bid Right Now" summary box, showing the driver's name, rating, and completion history. Below it, the "All Bids" list ranks all other drivers based on bid, displaying their profile avatars, star ratings, submitted amounts, color-coded status badges.

4. **Accepting the best offer:** the user can click the large green "Accept Best Bid Now" button in the left control panel or the "Accept" button inside the top-right pinned card to instantly award the contract to the top driver and close the auction.
5. **Extending the duration:** the user can click the "Extend Deadline" button to add more time to the auction clock and adjust the background scheduling.
6. **Modifying the cap:** the user can click the "Edit Cap Price" button to dynamically alter the maximum payout threshold for the route.
7. **Canceling the job:** the user can click the red "Cancel Auction & Remove Load" button to immediately terminate the live tracking and unpublish the listing.
8. The current price updates automatically on the screen whenever the backend engine pushes a price escalation event.
9. The tracker slider head moves dynamically across the bar to visually mirror the real-time shifting price state.
10. The live bids list updates, appends, and re-ranks driver rows instantly as new preferred bids are submitted to the server.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Current Bids Table	/auctions/:loadId/bids	GET	Build a SSE tunnel to subscribe all the bids related to this loadId
Load Spec Related Sections	/loads/:loadId	GET	Get a single load detail
Accept button	/auctions/:loadId/bids/:bidId	PATCH	Accept the bid
Cancel Auction Button	/auctions/:loadId	DELETE	Cancel the ongoing Auction
Extend Deadline Button	/auctions/:loadId	PATCH	Edit the auction deadline
Edit Cap Price Button	/auctions/:loadId	PATCH	Edit the auction cap price
Breadcrumb 'Active Loads'	N.A.	N.A.	Navigate back to 'Active Loads' page
Current Price	/auctions/:loadId/price	GET	Build a SSE tunnel to subscribe current price

Slider(share the same tunnel with Current Price)	/auctions/:loadId/price	GET	Build a SSE tunnel to subscribe current price
--	-------------------------	-----	---

Driver Facing Auction/Load Details Page

This is the load and auction details page viewed by trucker. The truckers can see the starting point and the destination of a load, estimated time and distance, truck requirement and current price on the left panel. They can also see a list of available bids and a route on the right panel. They can submit a bid and also accept the price.

Load Information: GREENLEAF, N#1234789, Posted 2h ago. Status: Refrigerated Van (Active). FROM: Newark, NJ (Jun 12, 09:30) TO: Chicago, IL (ORD) (Jun 14, 14:00). DISTANCE: 620 mi, 50h drive.

Load Specifications:

Truck Type: Refrigerated Van	Size: 53 ft	Weight: 55,000 lbs
Commodity: Frozen Produce	Certifications: Reefer HACCP	Driver Assist: Not Required

Live Auction: LIVE
Current Price: **\$847**
[Accept \\$847 Now](#)
or submit a higher bid
Your Bid Amount: [Place Bid](#)
⚠ Bids cannot be withdrawn after submission

Route Preview: Newark, NJ → Chicago, IL. 620 mi - 50h.

Current Bids: 3 bids

Driver	Bid Amount	Status	Time
Alex M.	\$1,050	Pending	2m ago
Sam K.	\$980	Pending	15m ago
Raj P.	\$850	Outbid	1h ago

Notification: You are about to bid \$1,050. This bid cannot be withdrawn. Deal auto-closes when price reaches your bid.

How to use:

- Viewing load information and specifications:** the top-left area displays the company name, unique load ID, active status badge, elapsed time since posting, vehicle type requirements, trailer dimensions, total freight weight, cargo commodity type, necessary certifications, and target route details.
- Tracking the live price point:** the left panel prominently features the "Current Price" display, showcasing the live dollar amount currently being offered to haul the load.
- Reviewing the route preview map:** the top-right section features a "Route Preview" map dashboard grid displaying the starting origin pin, the final destination point, and an overarching timeline route line summarizing the total distance and estimated driving duration.
- Monitoring active driver bids:** the lower-right area presents a "Current Bids" list displaying all submitted preferred bids from rival couriers. Each line displays the driver's name, their submitted bid amount, a color-coded status badge indicating standing position and an elapsed time tracker showing when the bid was placed.

5. **Claiming the load instantly:** the user can click the large blue "Accept \$847 Now" button in the left panel to instantly claim the delivery contract at the currently displayed marketplace system price.
6. **Submitting a preferred bid:** the user can type a specific target amount into the "Your Bid Amount" input field and click the black "Place Bid" button to register their preferred auto-close threshold.
7. The current price updates automatically on the screen whenever the backend engine pushes an algorithmic price-creep escalation event.
8. The live bids list updates, appends, and re-ranks driver rows instantly as competitor preferred bids are submitted to the server.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Load Spec Table	/loads/:loadId	GET	Get a single load detail
Place Bid Button	/auctions/:loadId/bids	POST	Place a bid for the load
Accept Price Button	/auctions/:loadId/claim	POST	Claim the load with current price immediately
Current Price	/auctions/:loadId/price	GET	Build a SSE tunnel to subscribe current price
Map - single route	/loads/:loadId/route	GET	Fetch a single route
Current Bids Table	/auctions/:loadId/bids	GET	Build a SSE tunnel to subscribe all the bids related to this loadId
Breadcrumb	N.A.	N.A.	Navigate to 'Loads Available' page

Driver Facing Profile Page

This is the profile page a driver will see. It will allow drivers to update their contact information, trucks, pricing preferences, and notification preferences. Additionally, truckers can specify if they are available for loads (which will act as a notification “switch”, disabling work-related notifications). Lastly, drivers will be able to see their own performance and review metrics.

The screenshot displays the 'My Profile' page for Alex Martinez, an Owner-Operator Driver. The page is organized into several sections:

- Profile Overview:** Includes a profile picture, name (Alex Martinez), title (Owner-Operator Driver), membership date (March 2024), a 4.9 star rating from 127 loads, and an 'Available for loads' toggle switch.
- Contact Information:** Fields for email (alex.martinez@email.com), phone ((604) 555-0123), and location (Vancouver, BC, Canada).
- Performance:** Metrics showing 127 Completed Loads, a 4.9 Avg Rating, a 98% On-Time Rate, and \$0 Disputes.
- My Truck:** A section for managing vehicles, currently showing a Peterbilt 389 - 2021 Refrigerated Van with a capacity of 53 ft and 55,000 lbs. It includes options to 'Delete Truck' or mark it as 'Primary'.
- Pricing Preferences:** Input fields for Minimum Rate (\$3.50 / mile), Minimum Load Value (\$1,500), and Preferred Max Deadhead (50 miles).
- Notification Preferences:** A list of notification types with corresponding toggle switches: New loads matching my filters (on), Price escalation alerts (on), Bid accepted / rejected (on), Delivery reminders (off), and Weekly earnings summary (on).

A 'Save Changes' button is located in the top right corner of the profile section.

How to use:

1. **Profile Overview:** The top-left panel shows your driver identity, rating, membership date, and an “Available for Loads” toggle. Switch this on or off to control whether you will receive notification information.
2. **Editing Contact Information:** Click any field (email, phone, location) to edit it. After making changes, press Save Changes to update your profile.
3. **Managing Truck Details:** The *My Truck(s)* section lists your vehicle(s)’s specification(s). Click any field (e.g., trailer type, capacity, certifications) to update your truck information, then save. Numerous trucks can be saved.
4. **Delete Truck:** Clicking the button will spawn a dialog asking the driver if they want to remove the truck from their fleet.
5. **Adjusting Pricing Preferences:** Enter your minimum rate per mile, minimum load value, and maximum deadhead distance. These settings influence which loads you are matched with and which bids you can place.

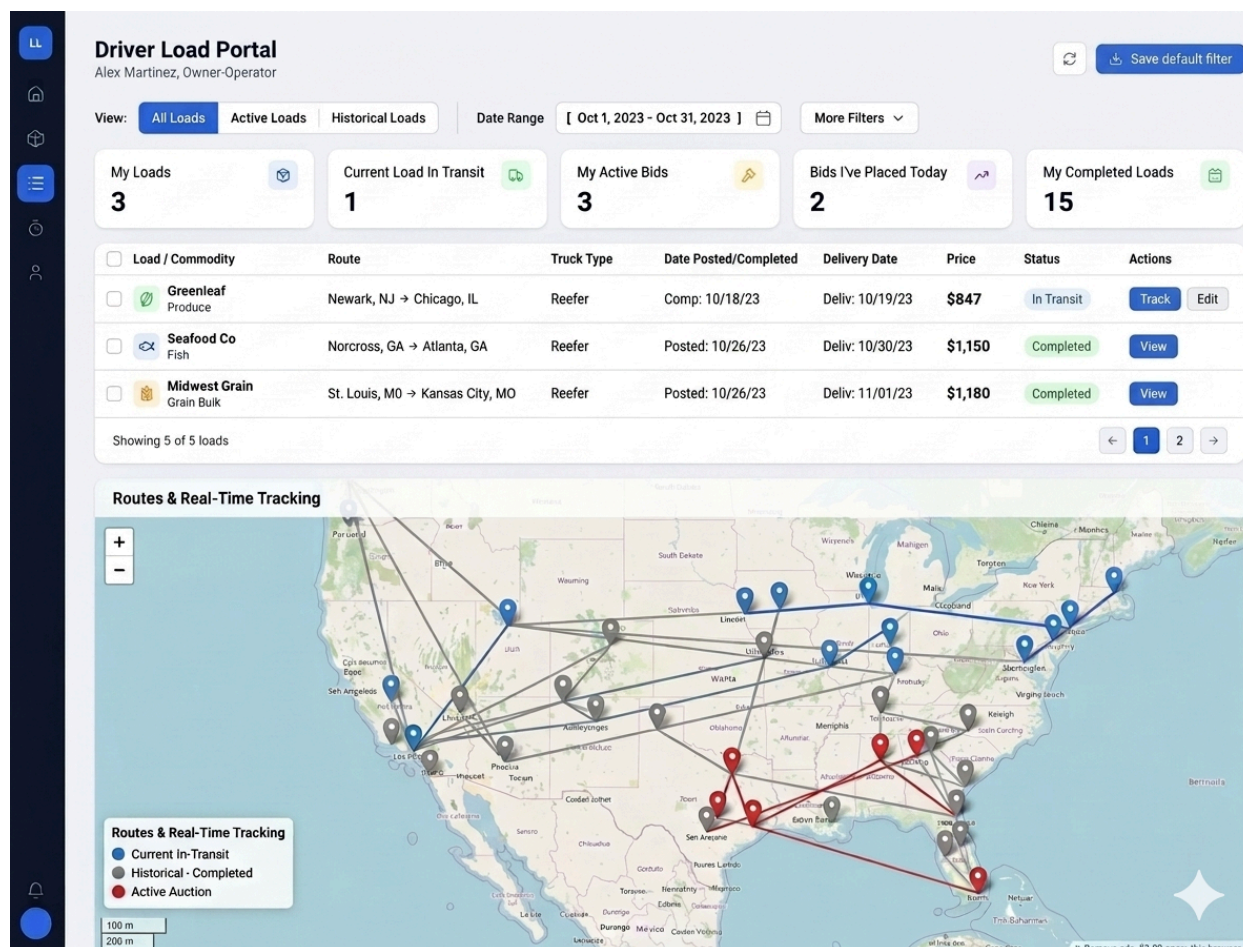
6. **Notification Preferences:** Toggle on/off alerts such as new matching loads, bid updates, delivery reminders, and weekly summaries. These determine what updates you receive from the platform.
7. **Saving Changes:** After modifying any section, click Save Changes in the top-right corner. All updated fields are sent to the backend in a single request.
8. **Viewing Performance Metrics:** The performance panel shows completed loads, average rating, on-time percentage, and disputes. These values are read-only and updated automatically based on your activity.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Availability toggle	/drivers/:id	PATCH	Update availability status
Contact information fields	/drivers/:id	PATCH	Update email, phone, location
Pricing preferences	/drivers/:id	PATCH	Update min rate, min load value, preferred max deadhead
Notification toggles	/drivers/:id	PATCH	Update notification settings
Fetching profile, preferences, notifications	/drivers/:id	GET	Load all driver profile data
Performance metrics panel	/drivers/:id/trucks	GET	Load read-only stats, such as completed # of loads, rating(s), etc.
Truck details (make/model/year, trailer type, capacity, certifications, plate)	/drivers/:id/trucks	GET	Load truck information
Editing truck details	/drivers/:id/trucks	PATCH	Update truck(s) specifications
Add truck details	/drivers/:id/trucks	POST	Add an additional truck to the driver's fleet
Delete truck	/drivers/:id/trucks	DELETE	Delete a truck from the driver's fleet

Driver Facing Dashboard

Dashboard the driver will see which provides a high-level view of their current and historical loads, as well as their active auction(s), loaded onto a map as a visual summary.



How to use:

- Dashboard Summary Cards:** The top row shows quick metrics such as My Loads, Current Load In Transit, My Active Bids, Bids Placed Today, and Completed Loads. These update automatically and give the driver a snapshot of their activity. Computed via the data provided in the load table.
- Load Table:** Each row represents a load the driver is involved with — either in transit, completed, or currently in auction. The driver can click Track, Edit, or View depending on the load's status.
- Tracking Loads:** Clicking Track opens real-time route progress for the in-transit load.
Stretch Goal
- Editing Loads:** If the load is still active or in auction, the driver can click Edit to adjust bid amount (ie., lower their bid from \$900 to \$800) or update notes (ie., “reminder to back into this company’s loading bay”).

5. **Viewing Completed Loads:** Clicking View opens a read-only summary of the completed shipment.
6. **Map Panel:** The map shows all relevant routes (colours/design slated to change):
 - a. Blue → current in-transit
 - b. Gray → historical completed
 - c. Red → active auctions
7. Clicking a route or marker opens (highlights) the corresponding load details.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Load list table	/drivers/:id/loads	GET	Fetch all loads associated with the driver (active, in transit, completed, auction)
Load row details	/loads/:loadId	GET	Fetch full details for a specific load
View button (completed loads)	/loads/:loadId	GET	Load read-only shipment summary
Edit button (active/auction loads)	/loads/:loadId	PATCH	Update editable load fields (bid amount, notes)
Map route overlays (in-transit, completed, auction)	/loads/:loadId/route	GET	Fetch all route polylines + statuses for map rendering
Clicking a route marker	/loads/:loadId	GET	Highlights the load in the table & view read-only shipment summary

Login page

This is the primary sign up/login page for the platform. There are two distinct user groups: drivers and companies. Companies are the ones posting the loads and accepting bids, etc. Drivers are the ones bidding, and driving the loads from origin to destination.

LoadLink

Connect. Bid. Deliver.

The open marketplace replacing freight dispatchers. Drivers bid directly, companies save — no middlemen.

2,400+ Active Drivers **840** Loads Today **\$0** Dispatcher Fees

Live load tracking - Powered by OpenStreetMap

Get started

Choose your role to continue

I'm a Driver
Browse loads, bid on jobs, track your routes

Selected

I'm a Company
Post loads, manage auctions, track deliveries

Email address

you@company.com

Password

[Forgot password?](#)

Sign In as Driver →

or

Continue with Google

Don't have an account? [Sign up free](#)

How to use:

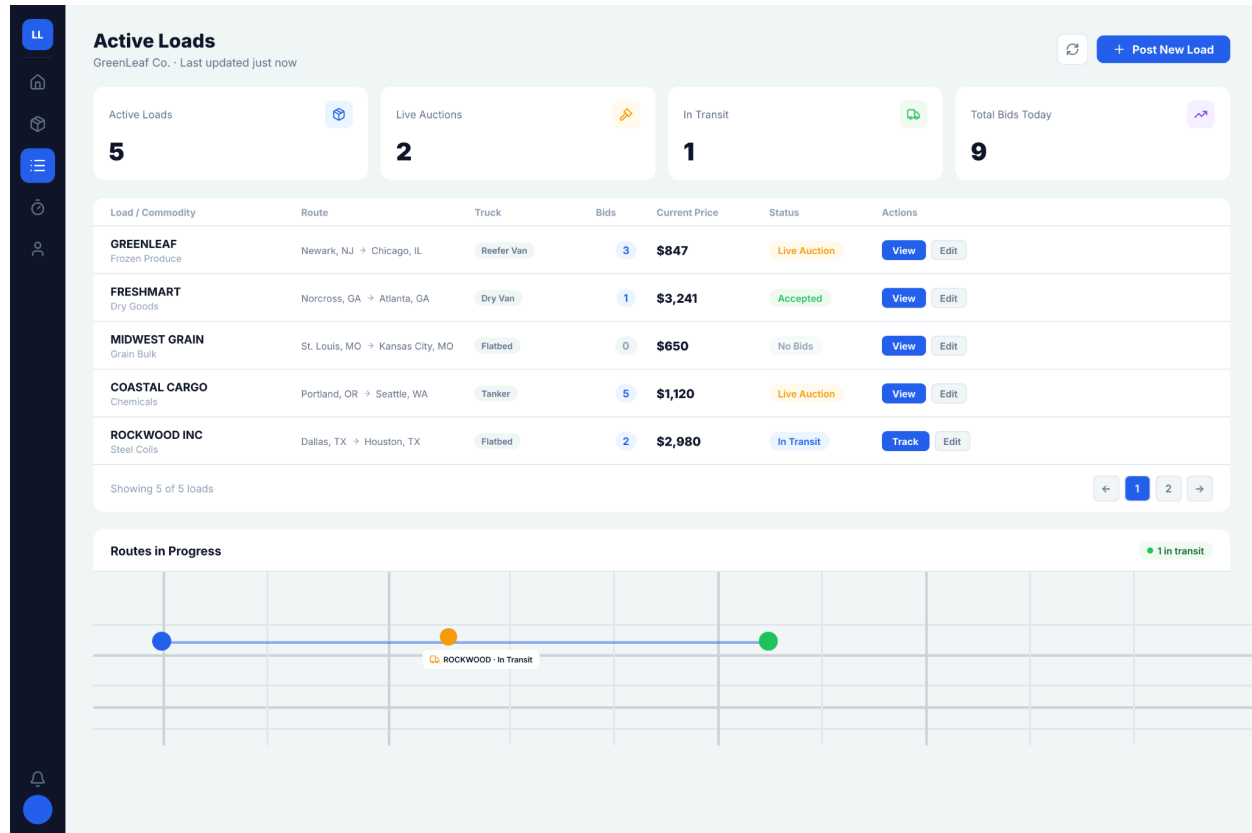
- Select a role:** the user can choose and select between 2 roles: a) Driver (browse loads, bid on jobs, track routes) or b) Company (post loads, manage auctions, track deliveries) and the selected card highlights in blue
- Enter credentials:** the user types an email address into the “Email Address” field and types a password into the “Password” field.
- Sign in:** the user clicks the “Sign in as [role]” button to authenticate.
 - Success: the user is redirected to the role specific dashboard (Driver home or Company home)
 - Failure (wrong credentials): an inline error message appears below the email/password fields
- SSO:** Click “Continue with Google” to authenticate via Google OAuth. Role selection still applies before OAuth redirects
- Sign up:** the user clicks on “Sign up free” to navigate to the registration flow
- Forgot password:** the user clicks on this button to be redirected to the password reset flow

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Sign in as Driver/ Company button	/auth/login	POST	Authenticates the user with email, password, and selected role. Returns a JWT if successful
Continue with Google button	/auth/google	GET	Redirects the Google OAuth and passes the selected role as query param
OAuth provider callback	/auth/google/callback	GET	Receives the OAuth code from Google, redirects to dashboard
Forgot password button link	/auth/forgot-password	POST	Sends password reset email to the submitted address

Company Facing Dashboard

This is the view that companies will see when they have jobs being completed or live auctions in progress. They can see a brief overview of their ongoing jobs/auctions and track the progress of each load.



How to use:

1. **Post New Load:** This will lead to the posting new load screen for companies
2. **Dashboard Summary Cards:** The top row shows quick metrics such as Active Loads, Live Actions, Load In Transit, and Total Bids Received Today. These update automatically and give the companies a snapshot of their activity. Computed via the data provided in the load table.
3. **Load Table:** Each row represents an active load the company has posted. The loads are currently in auction, in transit, or accepted. The company can edit the specifics of a load if not in transit or accepted.
4. **View Load:** Companies can view the details of load posting for any load
5. **Edit Load:** If the load is still in auction, the company can click Edit to adjust price cap or other load posting details
6. **The Map:** the map keeps track of loads currently in progress

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Load list table	/companies/:id/loads	GET	Fetch all loads associated with the company (active, in transit, auction)
Load row details	/loads/:loadId	GET	Fetch full details for a specific load
View button (completed loads)	/loads/:loadId	GET	Load read-only shipment summary
Edit button (active/auction loads)	/loads/:loadId	PATCH	Update editable load fields (price cap, description, etc)
Map route overlays (in-transit routes)	/loads/:loadId/route	GET	Fetch all route polylines for map rendering
Clicking a route marker	/loads/:loadId	GET	Highlights the load in the table & view read-only shipment summary

Company Facing New Load Posting Page

This is the screen that the freight companies will be using to post a new load for the drivers to bid on. The companies will specify load type, type of truck needed to complete the drive, auction price settings, etc.

They will be able to see a summary of their post before making the post public.

Post a New Load
Fill in the details below to post your load and start the reverse auction.

Truck & Load Details

Truck Type: Select type... Truck Size: e.g. 53 ft. Number of Trucks: 1

Weight (lbs): e.g. 40,000 Commodity Type: e.g. Frozen Produce, Steel Coils... Special Certifications: None (optional)

Driver Assist Required: Driver must help with loading/unloading at origin or destination

Route

Origin (Postal/ZIP Code): 07102 — Newark, NJ → Destination (Postal/ZIP Code): 60601 — Chicago, IL

+ Add stop (multi-destination)

Pickup Date: Jun 12, 2025 Pickup Time: 09:30 AM Delivery Date: Jun 14, 2025 Delivery Time: 02:00 PM

Pricing & Auction Settings

Minimum Price (Start): \$ 800 Maximum Price (Cap): \$ 1,000 Escalation Rate: \$ 10 / hour

Auction starts at this price Auction ends at this price How fast price auto-increases

Auto-Accept Tolerance (%): 10 % above max price Trigger (hours before deadline): 1 hour before deadline Additional Notes: e.g. fragile cargo, dock access required...

Will auto-accept lowest bid within tolerance if unbooked within 1 hr of deadline System auto-accepts the best eligible bid at this threshold

Load Summary

Company: GreenLeaf Co.
Truck: Refrigerated Van - 53 ft.
Route: Newark, NJ → Chicago, IL
Distance: ~820 mi - 90h
Weight: 55,000 lbs
Commodity: Frozen Produce
Certifications: Reefer HACCP
Driver Assist: Not Required
Pickup: Jun 12 - 09:30 AM
Delivery: Jun 14 - 02:00 PM

Auction Price Range
\$800 → \$1,000
+\$10/hr escalation - 10% tolerance

Post

How to use:

- Enter details:** The information needed for a load posting is grouped into categories into its own cards and the users can use one of text fields, dropdown, datepicker, etc to write their post
 - Truck & Load details: specify truck type, load type, etc
 - Routes: specify pickup location, dropoff location(s), dates, etc
 - Pricing & Auctions: set starting price, ending price, escalation rate of reverse auction, tolerance, etc.
- Load Summary:** Summary of the information entered by the user, this field is view-only
- Posting Load:** Use the “post” button to make the posting public for drivers to bid on

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Post button	/loads /companies/:id/loads/	POST	Create load with fields (price cap, description, etc) Add load ID to the list of company's loads

Driver Facing Load Board

This is the primary screen that drivers will see and interact with in relation to loads. They can view and manage their current job while also being able to look for their next loads using the search bar and filter options which can help narrow down jobs by distance or payout. Using the map and route timeline, they can see the route for each load, including recommended rest stops and fuelling. The filtered search loads section/recommended loads use their truck profile to suggest cargo nearby based on their historical jobs.

The interface is divided into three main sections:

- Left Panel (Filters and Load List):**
 - Search and Filters:** Includes a search bar for "Search loads, companies..." and filter pills for "300 mi", "Newark, NJ", and "Dry Van".
 - Active Bids:** A yellow banner indicates "2 active bids — accepting a load will auto-cancel them".
 - Recommended Loads:**
 - GREENLEAF:** WMS Newark, NJ to ORD Chicago, IL. Distance: 50mi, 620mi. Payout: \$4,500. Vehicle: Refrigerated Van.
 - FRESHMART:** WMS Norcross, GA to ATL Atlanta, GA. Distance: 40mi, 540mi. Payout: \$3,241. Vehicle: Dry Van.
 - Other loads:**
 - SILVERLINE:** DAL Dallas, TX to HOU Houston, TX. Payout: \$2,980. Vehicle: Dry Van.
- Shipment route overview:**
 - Distance: 620 mi · Est. 50h.
 - Map showing the route from Newark, NJ to Chicago, IL with a fuel stop.
 - Buttons: Route, Fuel Stops, Rest Areas.
- Delivery timeline:**
 - Visual timeline with icons for Pickup, Rest Stop, Sleep Break, Fuel Stop, and Delivery.
 - Table below:

Stage	Time	Location	Action
Pickup	Jun 12, 09:30	Newark, NJ	Start
Rest Stop	Jun 12, 14:00	Rest Area, PA	Break
Sleep Break	Jun 12, 22:00	Truck Plaza, I-95	Rest
Fuel Stop	Jun 13, 06:00	Pilot Travel Center	Refuel
Delivery	Jun 14, 14:00	Chicago, IL (ORD)	Complete

How to use:

- Search and filter (optional):** Specify various filters for loads you want to see (ex. Distance, origin, destination, truck type, rate, dates, etc.). Active filters appear as pills below the search bar.
- Browse the feed:** The left panel divides available loads into “Recommended” and “Other” where recommended loads are those that are based on your previous loads, preferences, etc, and other loads are all others that satisfy your filter.
- Selecting loads:** Clicking on a specific load’s card will bring you to the “Auction/Load Details Page - Trucker” page where the information for that load is posted, and you can see everything from the route details, to the other bids, to your bid settings, where you can place a bid, accept the current price, or go back to this page.

4. **Viewing the route of a load:** The overview maps the route of the load using Leaflet/MapBox. The user can toggle the route, fuel, and rest stops to appear on the map as well.
5. **Checking the timeline of a load:** The bottom right panel breaks down the route in a chronological order. It includes a timeline and a table that shows the pickup and dropoff, along with recommended rest stops and fuelling. This all helps the driver understand the load more.
6. **Managing active bids:** A little banner below the search bar and filters tells the driver if they have any active bids. If a driver accepts a load directly, or wins a bid, all other bids will be cancelled that interfere with that load that you won.
7. **Accepting a load:** When a driver wants to accept a load directly (ie they are happy with the price, etc), they can directly accept the load for the current price directly on the load card to claim the load.

API Elements used by this Screen:

UI Element	API Endpoint	Method	Purpose
Search and filter bar	/loads?radius=INT&origin=PLACE...	GET	Fetches a list of available loads that is dynamically filtered by the user specific parameters
Active bids warning banner	/drivers/:id/bids?status=active	GET	Fetches the drivers current active bids to display a warning banner
Load cards	/drivers/:id/recommended-loads	GET	Fetches the list of suggested loads tailored to the driver based on historical jobs, etc.
Load route map	/loads/:loadId/route	GET	Fetches the coordinate polylines and the geospatial data needed to create the visual map for the selected load
Delivery timeline	/loads/:loadId/timeline	GET	Fetches the chronological breakdown of the route including things like times for pickup and dropoff, fuelling, rest, etc.
Accept button	/auctions/:loadId/claim	POST	Instantly claims the load at the current auction price, closing the auction and generating the contracts

RESTful API Specification

This section outlines a high-level overview of the types of inputs and outputs we believe we will need for our web application. Note: to come up with this list, we took our wireframe section above, which includes descriptions of how the page works and what API calls we believe are necessary, and asked AI to come up with Request and Response schemas for how the data could be sent or received. We understand our schemas could very likely require change as we start to develop the project.

API name	Description	Request	Response
Login			
POST /auth/login	Authenticates a user (driver or company)	{ "email": "user@example.com", "password": "password", "role": "driver company" }	{ "token": "eyJhbGciOiJSUzI1NiIsInR5cCI6...", "expiresIn": 3600, "user": { "uid": "fb_uid_12345", "name": "Alex Martinez", "email": "user@example.com", "role": "driver", "profileComplete": true } }
GET /auth/google	Initiates Google OAuth and redirects to Google consent screen	Header: { "Authorization": "Bearer <firebase_auth_token>" } Query: role=driver role=company	302 Redirect to Google OAuth consent page
GET /auth/google/callback	OAuth callback. Exchanges authorization code for a token and redirecting to client	Query: { code: "<auth_code>", state: "<state>" } (supplied by Google)	302 Redirect → /dashboard (JWT set in cookie) 400 Redirect → /login?error=oauth_failed
POST /auth/forgot-password	Sends a password reset link to the provided email address	{ "email": "user@example.com" }	{ "message": "If that email exists, a reset link has been sent." }
POST	Resets the user's	{	{

/auth/reset-password	password using the token from reset email	"token": "reset_token_abc123", "newPassword": "NewSecurePass1!" }	"message": "Password reset successfully." }
Company Side Auction			
GET /auctions/:loadId/ bids	Build a SSE tunnel to subscribe all the bids related to this loadId	Header: { "Accept": "text/event-stream", "Authorization": "Bearer <firebase_auth_token>" } (no body)	{ "loadId": "665e3cb73f1d2a123456789a", "bids": [{ "bidId": "bid_002", "driverUid": "fb_uid_67890", "driverName": "Alex Martinez", "driverRating": 4.9, "bidAmount": 1050.00, "currency": "CAD", "timestamp": "2026-06-03T21:32:00Z" }, { "bidId": "bid_001", "driverUid": "fb_uid_55443", "driverName": "Sam Kowalski", "driverRating": 4.7, "bidAmount": 980.00, "currency": "CAD", "timestamp": "2026-06-03T21:30:00Z" }], "loadEventType": "ACTIVE" }
GET /loads/:loadId	Get a single load detail	(no body)	{ "loadId": "665e3cb73f1d2a123456789a", "companyId": "company_uid_5544", }

			<pre> "companyName":"GREENLEAF ", "status":"AUCTION_ACTIVE", "postedAt":"2026-06-03T19:49:5 1Z", "origin":{ "address":"Newark, NJ", "lat":40.7357, "lng":-74.1724 }, "destination":{ "address":"Chicago, IL (ORD)", "lat":41.9742, "lng":-87.9073 }, "routeSpecifications":{ "distanceMi":620, "estimatedDurationHours":50 }, "loadSpecifications":{ "truckTypeRequired":"53ft Refrigerated Van", "weightLbs":55000, "commodity":"Frozen Produce", "certifications":["Reefer HACCP"], "driverAssist":false } } </pre>
PATCH /auctions/:loadId/ bids/:bidId	Accept the bid	<pre> { "status": "ACCEPTED" } </pre>	<pre> { "loadId": "665e3cb73f1d2a123456789a", "finalPayout": 1050.00, "rateConfirmationUrl": "https://S3.amazonaws.com/loadl ink-docs/confirmations/rc_99887 7.pdf" } </pre>

DELETE /auctions/:loadId	Cancel the ongoing auction	(no body)	204 No Content
PATCH /auctions/:loadId	Update editable auction fields (deadline extensions, max price ceilings)	{ "extendByMinutes": 134, "newPriceCeiling": 1200.00, "autoAcceptTolerancePercentage": 10 }	{ "loadId": "665e3cb73f1d2a123456789a", "newExpiresAt": "2026-06-04T01:14:00Z", "priceCeiling": 1200.00, "autoAcceptToleranceThreshold": 1320.00 }
GET /auctions/:loadId/price	Build a SSE tunnel to subscribe current price	Header: { "Accept": "text/event-stream", "Authorization": "Bearer <firebase_auth_token>" } (no body)	{ "loadId": "665e3cb73f1d2a123456789a", "currentPrice": 847.00, "currency": "CAD", "loadEventType": "ACTIVE", "updatedAt": "2026-06-03T22:49:51Z" }

Trucker Side Auction

POST /auctions/:loadId/bids	Place a bid for the load	{ "bidAmount": 1050.00 }	{ "bidId": "bid_003", "loadId": "665e3cb73f1d2a123456789a", "bidAmount": 1050.00, "status": "PENDING", "message": "Bid successfully placed." }
POST /auctions/:loadId/claim	Claim the load with current price immediately	{ "claimPrice": 847.00 }	{ "loadId": "665e3cb73f1d2a123456789a", "finalPayout": 847.00, "status": "BOOKED", "rateConfirmationUrl":

			"https://s3.amazonaws.com/loadli nk/rc_998877.pdf" }
GET /loads/:loadId/route	Fetch a single route	(no body)	{ "loadId": "665e3cb73f1d2a123456789a", "distanceMi": 620, "estimatedHours": 50, "origin": { "lat": 40.7357, "lng": -74.1724 }, "destination": { "lat": 41.8781, "lng": -87.6298 }, "polylines": [[40.7357, -74.1724], [40.8000, -75.0000], [41.8781, -87.6298]] }
GET /loads/:loadId/timeline	Fetch the timeline for a route	(no body)	{ "loadId": "665e3cb73f1d2a123456789a", "timeline": [{ "stage": "Pickup", "time": "Jun 12, 09:30", "location": "Newark, NJ", "action": "Start" }, { "stage": "Fuel Stop", "time": "Jun 13, 06:00", "location": "Pilot Travel Center", "action": "Refuel" }, { "stage": "Delivery", "time": "Jun 14, 14:00", "location": "Chicago, IL (ORD)", "action": "Complete" }] }

GET /drivers/:id/recommended-loads	Fetch the recommended routes for a driver	(no body)	{ "driverId": "driver_123", "recommendedLoads": [{ "loadId": "665e3cb73f1d2a123456789a", "companyName": "GREENLEAF", "origin": "Newark, NJ", "destination": "Chicago, IL", "distanceMi": 620, "weightLbs": 55000, "currentPrice": 847.00 }] }
Blocklist & Preferences			
GET /users/:userId/blocklist?type=company and ?type=driver	Returns the user's blocklist, optionally filtered by companies or driver	Header: { "Authorization": "Bearer <firebase_auth_token>" } Query: type=company type=driver (optional) page=1, limit=20	{ "data": [{ "blockId": "block_001", "entity": { "id": "company_uid_5544", "name": "FastCargo LLC", "type": "company", "avatarUrl": null }, "reason": "Consistently low offers below my minimum rate.", "blockedAt": "2025-05-03T00:00:00Z" }], "pagination": { "page": 1, "limit": 20, "total": 2 } }
POST /users/:userId/blocklist	Adds an entry to the blocklist/ blocks them	{ "userId": "company_uid_5544", "userType": "company driver", "reason": "Posted load with inaccurate weight info." }	{ "blockId": "block_003", "user": { "id": "company_uid_5544", "name": "QuickHaul Inc.", "type": "company", }

			<pre>"avatarUrl": null }, "reason": "Posted load with inaccurate weight info.", "blockedAt": "2026-06-04T10:00:00Z" }</pre>
DELETE /users/:userId/blocklist/{blockId}	Removes an entry from the blocklist/ unblocks them	Header: { "Authorization": "Bearer <firebase_auth_token>" } Path: userId, blockId	204 No Content
POST /reports	Submits a fraud/inaccuracy report for review	<pre>{ "reportType": "fraud inaccurate_load harassment other", "targetId": "company_uid_5544", "targetType": "company driver load", "description": "Company misrepresented load weight causing overload violation.", "evidenceUrls": ["https://s3.amazonaws.com/loadlink- docs/evidence/img_001.jpg"] }</pre>	<pre>{ "reportId": "report_00421", "status": "pending", "submittedAt": "2026-06-04T10:15:00Z" }</pre>

Trucker Profile Action

GET /drivers/:id	Load driver profile (contact info, availability, pricing, notifications, trucks).	(no body)	<pre>{ "id": "driver_123", "name": "Alex Martinez", "email": "alex@example.com", "phone": "+1 555-123-4567", "location": "Dallas, TX", "availableForLoads": true, "pricing": { "minRatePerMile": 2.25, "minLoadValue": 300, "maxDeadheadMiles": 75 }, "notifications": { "newMatchingLoads": true, "bidUpdates": true, </pre>
------------------	---	-----------	---

			<pre>"deliveryReminders": true, "weeklySummaries": false } }</pre>
PATCH /drivers/:id	Update driver profile (contact info, availability, pricing, notifications).	<pre>{ "availableForLoads": false, "pricing": { "minRatePerMile": 2.50, "maxDeadheadMiles": 50 } }</pre>	<pre>{ "message": "Profile updated successfully.", "updatedFields": ["availableForLoads", "pricing"] }</pre>
GET /drivers/:id/performance	Get performance metrics (read-only).	(no body)	<pre>{ "completedLoads": 142, "averageRating": 4.8, "onTimePercentage": 96, "disputes": 1 }</pre>
GET /drivers/:id/trucks	Get all trucks in the driver's fleet.	(no body)	<pre>[{ "truckId": "truck_001", "make": "Freightliner", "model": "Cascadia", "year": 2021, "trailerType": "Dry Van", "capacityLbs": 45000, "certifications": ["TWIC", "Hazmat"], "plate": "ABC-1234" }]</pre>
POST /drivers/:id/trucks	Add a truck.	<pre>{ "make": "Volvo", "model": "VNL 760", "year": 2022, "trailerType": "Reefer", "capacityLbs": 48000, "certifications": ["Hazmat"], }</pre>	<pre>{ "message": "Truck added successfully.", "truckId": "truck_002" }</pre>

		"plate": "XYZ-9876" }	
PATCH /drivers/:id/trucks /:truckId	Update truck details.	{ "trailerType": "Flatbed", "capacityLbs": 50000 }	{ "message": "Truck updated successfully.", "updatedFields": ["trailerType", "capacityLbs"] }
DELETE /drivers/:id/trucks /:truckId	Delete a truck.	(no body)	204 No Content

Company Profile Action

POST /companies/:id/loads	Add load ID to the list of company's loads	{ "originAddress": "07102 - Newark, NJ", "destinationAddress": "60601 - Chicago, IL", "truckTypeRequired": "Refrigerated Van", "weightLbs": 55000, "commodity": "Frozen Produce", "startPrice": 800.00, "maxPriceCap": 1000.00, "escalationRatePerHour": 10.00 }	{ "loadId": "load_uid_1234", "status": "AUCTION_ACTIVE", "message": "Load posted successfully." }
GET /companies/:id/loads	Loading list of loads that belong to the company	(no body)	{ "companyId": "company_uid_1234", "loads": [{ "loadId": "load_uid_1234", "commodity": "Frozen Produce", "status": "LIVE_AUCTION", "bidsCount": 3, "currentPrice": 847.00, "route": "Newark, NJ -> Chicago, IL" },]

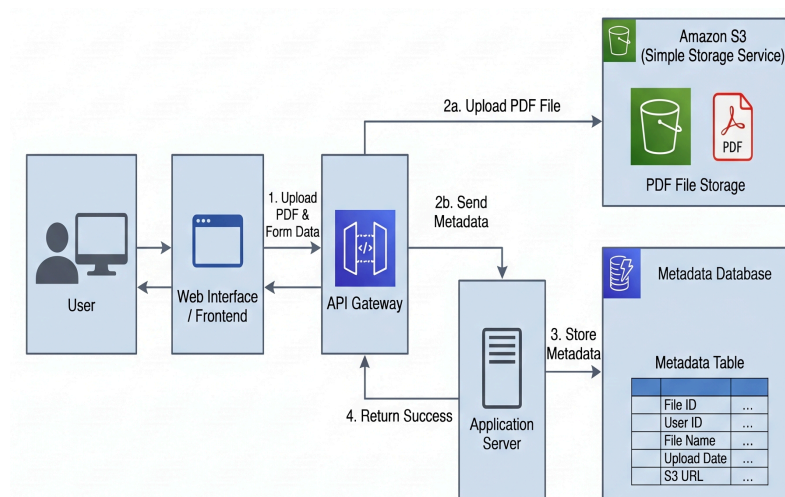
			<pre>{ "loadId": "load_uid_5678", "commodity": "Dry Goods", "status": "ACCEPTED", "bidsCount": 1, "currentPrice": 3241.00, "route": "Norcross, GA -> Atlanta, GA" }]</pre>
GET /loads/:loadId	Load row details	(no body)	<pre>{ "loadId": "load_uid_1234", "status": "LIVE_AUCTION", "currentPrice": 847.00, "origin": { "address": "Newark, NJ" }, "destination": { "address": "Chicago, IL" } }</pre>
PATCH /loads/:loadId	Update editable load fields (price cap, description, etc)	<pre>{ "maxPriceCap": 1100.00, "notes": "Fragile cargo, dock access required." }</pre>	<pre>{ "loadId": "load_uid_1234", "updatedFields": ["maxPriceCap", "notes"], "message": "Load successfully updated." }</pre>

Backend

File Data Stored in S3

The following is the flow of how truckers' certificates and contracts will be stored in our AWS S3 service. When file data comes in, we will store S3 url and other metadata into the database and store the actual file into S3.

Note: While driver credentials are manually uploaded via the React frontend, the legal Rate Confirmation contracts are automatically generated server-side using pdf-lib before being pushed through to AWS S3.



Third Party APIs and Services

Firebase Authentication API

This is used to handle secure user identities, registration, and session management. The client facing React will interface directly with Firebase for users to login, and the Node backend will use the Firebase Admin SDK to verify the signatures of the incoming JWTs. As a result, every request to create a load, place a bid, or upload documents will be verified and authorized before connecting to our backend.

AWS API

We will use AWS S3 for cloud object storage for somewhat large files like drivers license uploads, insurance documents, and load related Rate Confirmation documents. We will use the AWS SDK to securely upload and retrieve these documents. When a driver uploads their licenses, or when we create a Rate Confirmation, the Node backend will push the file to an S3 bucket with a secure API call. S3 would respond with a permanent, secure URL that our backend will save to our main database.

Leaflet API

Leaflet (which uses OpenStreetMap) will be our main open source mapping framework that we will use to display routes and coordinates on the frontend. Leaflet handles the client side rendering and we will use OSMs built in geocoders to be able to have the human readable addresses turn into coordinates. This allows drivers and companies to see their active load paths, pick up and drop off locations, and see their old routes all in an interactive map.

LLMs

We have only used LLMs for generating our initial UI mockups and will not be pushing AI generated code to production without proper review. In terms of LoadLink, we may integrate AI through API calls to OpenRouter to suggest rest areas and fueling stops on routes, however, that is a stretch goal. Another stretch goal is to integrate a custom ML model to power our “AI Price Suggestion” feature, which analyzes historical load data to recommend prices for both the drivers and the companies.

Database Information

We will be using MongoDB Atlas as our main data storage method. We chose MongoDB because the document based structure is very flexible and can handle the complex queries within our load matching and bidding systems. We will also use Mongoose ODM to enforce schemas across various pieces of data like user profiles, load details, bids, reviews, etc. MongoDB also lets us do geospatial querying so we can filter and match loads based on drivers locations more easily.

Regular Data Stored in Database

User Profiles (PII): Things like names, contact information, company details, etc.

Truck and Truck Driver Data: Truck dimensions, licensing information, driver certifications, reviews, etc.

Load Details: Origin, destination, pricing, weight, commodity type, etc.

Transaction Data: Bidding data, pricing details, revenue, expenses, etc.

Geospatial Data: GPS location/coordinates, routing, etc.

Transit and Compliance Documents: Auto-generated Rate Confirmations (contracts) upon auction completion, and image data for Bills of Lading (BOL) used for proof of delivery

Test Plans

While in a perfect world a testing coverage of 100% across lines, statements, branches, and functions would be ideal, we understand there are diminishing returns capturing the last few coverage points. We will strive for, at a minimum, 80% coverage across lines, statements, branches, and functions. Front-end testing can be difficult to capture every possible scenario effectively.

Front-end Test Plan

The focus will be on ensuring user experience and data integrity on the client side. Front-end testing can be notoriously difficult, so the focus will be more on attempting to mimic how a user would flow through the app (such as by testing clicks, and verifying the UI reacts as expected when certain events occur).

In our previous milestone we highlighted: component rendering, map rendering, input validation, role-based access, API mocking, navigation flow, and the (potential) use of SSE. These are all still true, and components will be tested (where applicable) from a unit-test perspective. From a high-level perspective, we anticipate the following main critical areas of focus for the systems in our front-end.

A. Authentication & Role-Based Access

Test cases

- Login form rejects invalid credentials
- Login success with valid Firebase Auth token
- Shipper dashboard hides trucker-only components
- Driver dashboard hides shipper-only components
- When trying to access a page and not logged in (or with invalid credentials), should either redirect to login page or 403 error page

Required Data

- Mock firebase Auth Responses
- Mock user roles, ie. { role: “driver” }, { role: “shipper” }

B. Dashboards

Test cases

- Load list renders with data from /drivers/:id/loads
- Summary statistics are correctly calculated from load data
- Empty state renders when no load exists (ie., either an error state or simply that no loads could be found)
- Clicking a load navigates to (or displays) content from /loads/:id
- Filters (equipment type, distance, price) which update the list
 - Map reflects only the visible (ie., unfiltered) loads
- Map correctly displays source/destination and its route
- Clicking on a route on the map will highlight in table and display information from that route
- Pagination or infinite scrolling (we have not decided yet), verify all data is still accessible

Required Data

- /loads/:id → Mock the full details of at least one load

- /drivers/:id/loads → Mock 5-10 loads of different status (in progress, historical, currently bidding on, etc)
- /loads/:loadId/route → Mock the coordinates of the source/destination from the load

C. Auction/Bidding Workflow (Drivers)

Test cases

- Bid input rejects non-numeric, negative, and zero values
- Big input value can be higher than current lowest bid (ie., if current bid is \$800, can input \$805. Reason: the \$800 bid may be unable to complete)
- UI updates when a new bid arrives via SSE
- UI updates when an auction ends (your bid is accepted)
- UI updates when an auction ends (other bid is accepted)
- Auction timer counts down correctly
- Current price ticks up correctly
- Successful bids update /drivers/:id/loads
- Late bids are rejected
- Auction correctly displays details from individual load

Required Data

- /auctions/:loadId/bids → Mock valid and invalid bids
- /auctions/:loadId/price → Mock the price increasing and being
- /drivers/:id/loads → Mock load for live auction
- /loads/:loadId → Mock details for load in individual auction

D. Auction/Bidding Workflow (Companies)

Test cases

- UI updates when a new bid arrives via SSE
- Auction timer counts down correctly
 - Time-remaining updates correctly
- Current price ticks up correctly
- UI updates when instantly accepting a bid, ending the auction
- UI correctly displays load information
- UI correctly displays auction information

Required Data

- /loads/:loadId → Mock details of a load
- /auctions/:loadId → Mock details of an auction
- /auctions/:loadId/bids → Mock array of valid bids

General Testing:

- Test toasts appear on appropriate actions when and only when they are called
- Test basic error handling and retry mechanics (400/500 errors)
- Test error states on content not loading correctly
- Test error states on empty states (ie., there is naturally no data available)
- Test loading states (likely with loading skeletons)
- Test save/submit/delete/edit buttons display loading indicators if/as applicable
- Test buttons are disabled when action cannot be performed
- Test input validation forces the correct values to be input if/as applicable

Testing the reverse auction system will be tricky. We will need a way to reliably mock, among other things, the progression of time as our auctions are on a “timer”.

Backend Test Plan

The focus will be on function/logic accuracy and security using tools like Mocha/Chai or Jest for integration testing of: authentication middleware, bidding logic, API load testing, reverse auction validation. There are cases that verify that the auctions close right when there is an exact price acceptance, that user bids that are lower than each other overwrite each other, rejecting higher bids, and that late bids are rejected.

A. Authentication & Role-Based Access

- Test cases
 - Request without an HTTP Authorization header returns a 401 Unauthorized status.
 - Request with an invalid or expired Firebase Auth token returns a 401 Unauthorized status.
 - Driver attempts to access company-only routes and receives a 403 Forbidden status.
 - Company attempts to access driver-only routes and receives a 403 Forbidden status.
 - Valid Firebase Auth token correctly decodes, extracts user context, and appends the appropriate role and ID to the request object.
- Required Data
 - Mocked Firebase Auth verification utility responses.
 - Mock database user records containing structured role properties: { role: "driver" }, { role: "company" }.

B. Load Management & Fleet Filtering API

- Test cases
 - /drivers/:id/loads successfully aggregates and returns active, in-progress, and historical loads matching the specific driver's history.
 - Creating a load via POST /companies/:id/loads validates all required schema properties (origin, destination, weight, dimensions, pickup/drop-off windows) and returns a 201 Created` status with the complete populated JSON object.
 - Creating a load fails and returns a 400 Bad Request if missing critical schema fields.
 - GET /companies/:id/loads correctly executes pagination, limit constraints, and text search parameters.
 - Fleet filtering parameters (equipment type, weight, certifications) accurately filter out matching database records via query constraints.
- Required Data
 - Seeded load collection with diverse operational statuses, weight values, and localized route vectors.
 - Fleet configuration records mapping valid vehicle characteristics to mock drivers.

C. Reverse Auction & Bidding Workflow (Drivers / Inbound)

- Test cases

- POST /auctions/:loadId/bids rejects non-numeric, zero, or negative bid amounts with a 400 Bad Request status.
- Placing a bid updates the specific auction record's tracking history array atomically without modifying the historical data of other active bids.
- Placing a bid could be accepted by company even if it is higher than the current system price, provided it complies with the company's maximum cap and tolerance configuration.
- A driver attempting to bid on a load posted by a company that has blacklisted them is blocked and receives a 403 Forbidden status.
- Driver clicks to instantly claim the delivery contract at the current price; the server verifies availability, marks the auction closed immediately, and flags subsequent incoming claims as rejected.
- Required Data
 - Active auction document featuring configured cap values, active company ownership flags, and structured bid tracking arrays.
 - Active company blacklist configuration mapping blocked driver identifiers.

D. Reverse Auction Lifecycle Engine (System / Outbound)

- Test cases
 - The centralized heartbeat query accurately selects only the active loads where the calculated nextPriceExtensionAt property is less than (because clock cannot hit an exact match) or equal to the current system time.
 - When a timer tick expires, the server increments the live current price by the preset step amount atomically, utilizing optimistic locking to prevent overlapping human writes.
 - The system accurately updates the nextPriceExtensionAt field to reflect the new time milestone window extended by the specified step configuration.
 - An update to an auction's price point automatically triggers a real-time event broadcast via our streaming pipeline (SSE) strictly targeting the isolated room context.
 - When the countdown timer hits absolute zero or the current price crawls past the set cap value, the engine marks the auction status closed, rejects late claims, and identifies the correct winning driver.
- Required Data
 - Mocked system clock configuration to accurately evaluate time-dependent criteria intervals without stalling integration executions.
 - Seeded records mapping precise historical price thresholds, incremental rate settings, and expiration milestones.

E. Company Administration API

- Test cases
 - PATCH /auctions/:loadId validates company ownership, extends the overall auction timeline window, resets the background scheduling engine milestones, and fires a broadcast notification.
 - PATCH /auctions/:loadId fails with a 400 Bad Request if the auction has already closed or reached termination rules.

- PATCH /auctions/:loadId updates the maximum allowance boundary and dynamically recalculates auto-accept tolerance logic.
- DELETE /auctions/:loadId changes the database state to canceled, immediately stops the heartbeat loop execution, and sends a termination alert to watching participants.
- Required Data
 - Valid active load records linked directly to authenticated corporate user accounts.

F. General Error, Security, & Load Performance Handling

- Test cases
 - Global database connection dropouts or timeout conditions are caught gracefully by database middleware and return standard 500 Internal Server Error responses instead of crashing the process.
 - Invalid URL routing parameters or malformed data string lengths return clean 400 Bad Request formatting blocks.
 - Input sanitization middleware strips away hazardous injection sequences or script tags from load specification fields.
 - Concurrent bid load performance testing: simulating 50 concurrent incoming driver requests at the exact same millisecond to evaluate that race conditions are eliminated and data remains fully synchronized.
- Required Data
 - Batched arrays of concurrent mock request schemas containing competing payload data configurations.

Task Assignments

Our team is experienced yet diverse, with many of us either having internship experience or have worked on numerous hackathon projects. While the responsibilities provide a high-level look of each member's current experience, each member may want to try new things for some period of time and we welcome the team trying different roles.

For this milestone, we are maintaining the same role distribution as our system right now is currently working for us.

Name	Role(s)	Responsibilities
Wendy Tso	FS/frontend/ PM	- Managing timeline, creation of project documentation, coordination of tasks - Build reusable UI components - Help with setting up frontend architecture and managing state for dashboard/bidding interfaces using React - Implement client-side form validation for load postings/user profiles
Max Yan	FS/backend/Infra	- Create a Github Action pipeline to run the regression tests after creating PR(TBD) - API design and implementation(auction, dashboard etc.) - Authentication - Backend testing - Help with creating front-end components
Kyle Jones	FS/frontend/QA testing	- Create frontend elements (components) - Design initial CSS variables for consistency - Write frontend testing framework & focus on testing coverage
Eojin Lee	FS/backend	- Design database schema for loads, reviews, users, etc - Backend testing using Jest to be run with each PR - Build responsive UI components - Backend logic for auction, filtering, etc
Alexandar Lackovic	FS/backend/PM	- API design and implementation - Backend logic (bidding, routing, load management, etc.) - Create frontend elements (components) - Writing a comprehensive backend testing framework that focuses on test coverage

